Lava — Agent Guide

> This file is intended for AI coding agents. It describes the project structure, build system, conventions, and things you need to know before modifying code.

INHERITED FROM constitution/AGENTS.md

All rules in `constitution/AGENTS.md` (and the `constitution/Constitution.md` it references) apply unconditionally. Lava-specific rules below extend them — they MUST NOT weaken any inherited rule. The HelixConstitution submodule was incorporated 2026-05-14 (29th §6.L cycle); see root `CLAUDE.md` §6.AD + §6.AD-debt for the implementation-gap inventory.

Code intelligence — codegraph

This repository is indexed by **codegraph** — a local SQLite semantic code-knowledge-graph tool exposed to AI agents over MCP, wired into all five supported CLI agents (Claude Code, OpenCode, Qwen Code, Kimi CLI, Crush). Prefer its `codegraph_search` / `codegraph_context` MCP tools over blind file scanning. Full details — install, per-agent wiring, and the anti-bluff verification suite `scripts/verify-codegraph.sh` — are in `docs/CODEGRAPH.md`. The agent instruction file for Qwen Code is `QWEN.md` (a plain-text pointer to `CLAUDE.md`), present at the repo root and in every submodule.

Project Overview

Lava is an unofficial Android client for rutracker.org (and rutor.info). It consists of three main artifacts:

1. **Android App** (`:app`) — a modular Android application written in Kotlin, using Jetpack Compose for the UI.
2. **Go API Service** (`lava-api-go/`) — a headless Go/Gin server (SP-2 onward) that scrapes rutracker.org and exposes a JSON REST API over HTTP/3 and HTTP/2. This is the primary backend.

3. **Proxy Server** (`:proxy`) — **REMOVED 2026-05-06.** The legacy Ktor/Netty proxy was deleted when the SP-2 migration made `lava-api-go` the sole backend. There is no `proxy/` directory, it is not in `settings.gradle.kts`, and there is no `--legacy` fallback. (`build_and_release.sh`, `docker-compose.yml`, and `start.sh` carry comments documenting the removal.)

A fourth artifact is in progress: the **Lava API Android app** (`:api-app`, applicationId `digital.vasic.lava.api`, debug `.dev`) — a standalone app that boots the full `lava-api-go` server **in-process** on a phone/tablet, backed by a local SQLite database, and exposes it on the LAN via mDNS. Phases A (additive SQLite storage backend, selected by `LAVA_API_STORAGE_BACKEND`; Postgres default unchanged), B (the `internal/mobile` `Start`/`Stop`/`Status` embed + the `go build -buildmode=c-shared` native build at `lava-api-go/scripts/build-cshared.sh` + JNI bridge `digital.vasic.lava.apigo.LavaNative`), C (the `:core:apiengine` Kotlin JNI wrapper — `ApiEngine`/`NativeApiEngine`/`FakeApiEngine` + the `buildCshared`→jniLibs→`externalNativeBuild` Gradle pipeline packaging `liblavaapi.so` + `liblavaapi_jni.so` per ABI; commit `ef109760`), and D-infra (the `:api-app` module: foreground `ApiEngineService` with Wifi/Multicast/Wake locks, the Android-free `ApiEngineController` Stopped/Starting/Running/Stopping/Error state machine, the `NsdMdnsAdvertiser` emitting `_lava-api._tcp`/`_lava-api-dev._tcp` + TXT `engine/platform=android/storage=sqlite/version`, and the per-install `ApiKeyStore` base64-UUID credential in `EncryptedSharedPreferences`; commit `e62e0fa8`) have landed. The landing UI + control screen + ViewModel (D-ui), the instrumented Compose UI Challenge tests (E), and the client-side distinct labelling of Android instances (sub-project 2) are **PENDING**. See `docs/ON_DEVICE_API.md` and `docs/scripts/build-cshared.sh.md`. Note: `gomobile bind` is blocked on this module by the relative `replace ../submodules/*` directives — c-shared is the chosen path.

The project is a fork of `andrikeev/Flow`, maintained under `milos85vasic/Lava`. All source code, comments, and documentation are in **English**.

- **App ID:** `digital.vasic.lava.client`
- **App Version:** `1.2.0` (`versionCode = 1020`)
- **Proxy Version:** `1.0.3` (`versionCode = 1003`)
- **Go API Version:** `2.0.8` (`Code = 2008`)
- **License:** MIT (see `LICENSE`)

Technology Stack

Android / Kotlin Side

Layer	Technology
Language	Kotlin `2.1.0`
Build System	Gradle `8.9` (via wrapper)
Android Gradle Plugin	`8.6.1`
Android compileSdk	`35`
Android minSdk	`21`
Android targetSdk	`35`
JDK Toolchain	`17`
UI Framework	Jetpack Compose (BOM `2025.06.01`) — 100 % Compose, no XML layouts
State Management	Orbit MVI `7.1.0`
Dependency Injection (Android)	Dagger Hilt `2.54`
Dependency Injection (Proxy)	Koin `3.4.0`
Networking	Ktor client `2.3.1`, OkHttp, Coil `2.7.0`
HTML Parsing	Jsoup `1.15.3`
Database	Room `2.7.2` with KSP
Background Work	WorkManager `2.10.2`
Serialization	Kotlinx Serialization `1.6.3`
Crash Reporting	Firebase Crashlytics (via Firebase BOM `33.15.0`)
Debug Tools	LeakCanary `2.10`, Chucker `4.0.0`
Code Formatting	Spotless `6.22.0` with ktlint

Go API Side

Layer	Technology
Language	Go `1.25.0`
Web Framework	Gin Gonic `v1.12.0`
Transport	HTTP/3 (QUIC via `quic-go` `v0.59.0`) + HTTP/2 TLS fallback
HTML Scraping	goquery `v1.12.0`
Database / Cache	PostgreSQL via `pgx/v5`

Layer	Technology
Migrations	`golang-migrate/migrate` v4
Code Generation	`oapi-codegen v2.6.0`
Observability	Prometheus metrics, OTLP tracing, structured logging (`slog`)

Repository Layout

The repository is a **multi-module Gradle project** with an embedded Go service and Git submodules. There is **no root `build.gradle.kts`**; all Kotlin/Android build logic is centralized in `buildSrc` as custom convention plugins.

```
Lava/
├── app/                    # Android application module
├── lava-api-go/            # Go API service (SP-2 onward, sole
backend)
├── buildSrc/               # Custom Gradle convention plugins
├── core/                   # 25 core library modules
│   ├── auth/api, auth/impl
│   ├── common, data, database, designsystem, dispatchers, domain,
downloads, logger, models, navigation, notifications, preferences,
testing, ui
│   ├── network/api, network/impl
│   ├── tracker/api, tracker/client, tracker/mirror, tracker/
registry, tracker/rutor, tracker/rutracker, tracker/testing
│   └── work/api, work/impl
├── feature/                # 16 feature modules
│   ├── account, bookmarks, category, connection, favorites,
forum, login, main, menu, rating, search, search_input,
search_result, topic, tracker_settings, visited
├── submodules/             # vasic-digital submodules (frozen-by-
default pins)
│   ├── Auth, Cache, Challenges, Concurrency, Config, Containers,
Database, Discovery, HTTP3, Mdns, Middleware, Observability,
RateLimiter, Recovery, Security
│   └── Tracker-SDK/        # generic tracker-SDK primitives
(Gradle composite build)
├── gradle/
│   └── libs.versions.toml  # Version catalog
├── scripts/                # Local CI/CD apparatus (ci.sh,
tag.sh, bluff-hunt.sh, ...)
│   └── ...
├── .githooks/              # pre-push hook (Anti-Bluff + Local-
Only CI gate)
├── .lava-ci-evidence/      # Per-tag evidence packs (Sixth +
Seventh Law)
├── docs/                   # Architecture, design specs, release
```

```
notes, dev guide
├── build_and_push_docker_image.sh
├── build_and_release.sh
├── start.sh / stop.sh
├── Upstreams/              # Upstream repository push scripts
└── settings.gradle.kts
```

Consult `settings.gradle.kts` for the live module list. The exact count fluctuates as SP-N projects extract or fold modules.

Module responsibilities

- `:app` — Entry point. Contains `Application`, `MainActivity`, `TvActivity`, and the top-level navigation graph. Depends on every core and feature module.
- `:proxy` — **REMOVED 2026-05-06** (legacy Ktor/Netty server). Superseded by `lava-api-go`; no `proxy/` module remains.
- `lava-api-go/` — Go/Gin server exposing REST endpoints over HTTP/3 + HTTP/2. Sole backend. Built as a static binary and a distroless Docker image. Also hosts the on-device-API additions: `internal/storage/` (backend-agnostic cache boundary with `postgres` + pure-Go `sqlite` impls), `internal/mobile/` (the in-process `Start`/`Stop`/`Status` embed), and `cmd/lavaapi-cshared/` (the `go build -buildmode=c-shared` native entry + `jni/` bridge). See `docs/ON_DEVICE_API.md`.
- `core:*` — Shared libraries. Pure Kotlin modules (`models`, `common`, `auth/api`, `network/api`, `tracker:api`, `tracker:registry`, `tracker:mirror`, `tracker:rutracker`, `tracker:rutor`, `tracker:testing`, `work/api`) have **no Android dependency**. The Android-bearing tracker module is `:core:tracker:client` (Hilt + WorkManager + Room access).
- `:core:apiengine` — Android library wrapping the embedded `lava-api-go` server (a Go c-shared `.so` loaded over JNI) behind the Kotlin `ApiEngine` API (`NativeApiEngine` real impl + `FakeApiEngine` JVM fake). Consumed by `:api-app`; packages `liblavaapi.so` + `liblavaapi_jni.so` per ABI via a `buildCshared` task + `externalNativeBuild` CMake. Phase C of the on-device-API sub-project; see `docs/ON_DEVICE_API.md`.
- `:api-app` — standalone Lava API Android app (`digital.vasic.lava.api`, debug `.dev`); foreground service hosting the embed + mDNS advertiser + per-install auth-key store (Phase D-infra). See `docs/ON_DEVICE_API.md`.
- `feature:*` — Screen-level modules. Each feature typically contains a ViewModel (Orbit MVI), Compose screens, and a navigation contract.

SDK module map (added by SP-3a)

The multi-tracker SDK introduced by SP-3a sits across two locations:

Local (Lava-domain) Kotlin modules under `core/tracker/`:

Module	Purpose
`:core:tracker:api`	Lava-domain feature interfaces (`Searchable`, `Browsable`, `Topic`, `Comments`, `Favorites`, `Authenticatable`, `Downloadable`), `TrackerCapability` enum, common data model (`TorrentItem`, `SearchRequest`, `SearchResult`, etc.). Pure-Kotlin, JVM-only.
`:core:tracker:registry`	Lava-domain wrapper around the generic `lava.sdk:registry` primitive in the `Tracker-SDK` submodule. Discovers and exposes `TrackerClient` instances.
`:core:tracker:mirror`	Lava-domain wrapper exposing `MirrorConfigStore` typealias and bridging to `lava.sdk:mirror`.
`:core:tracker:client`	`LavaTrackerSdk` orchestrator — switches active tracker, runs cross-tracker fallback, persists user mirrors and mirror health. Hilt-injected entry point for feature ViewModels.
`:core:tracker:rutracker`	RuTracker-specific implementation. 12 declared capabilities — SEARCH + BROWSE + FORUM + TOPIC + COMMENTS + FAVORITES + DOWNLOAD + MAGNET + AUTH (CAPTCHA_LOGIN) + UPLOAD + USER_PROFILE (the latter two are LF-5 latent). Encoding: Windows-1251.
`:core:tracker:rutor`	RuTor-specific implementation. 8 declared capabilities — SEARCH + BROWSE + TOPIC + COMMENTS + DOWNLOAD + MAGNET + RSS + AUTH (FORM_LOGIN). Anonymous-by-default per decision 7b-ii. Encoding: UTF-8.
`:core:tracker:testing`	Test fakes (`FakeTrackerClient`, builders, fixture loaders) shared across tracker modules.

Generic (vasic-digital) primitives mounted at `submodules/tracker_sdk/`:

Submodule path	Purpose
`submodules/tracker_sdk/api`	

Submodule path	Purpose
	Generic feature-bearing primitives (`MirrorUrl`, `TrackerDescriptor`-shape, `Mirror` health states). Lava-agnostic.
`submodules/tracker_sdk/registry`	Generic in-memory registry of tracker clients.
`submodules/tracker_sdk/mirror`	Generic mirror-config store interface (`MirrorConfigStore`).
`submodules/tracker_sdk/testing`	Generic test scaffolding (clock, dispatcher, fixture loader).

The submodule is **frozen by default** per the Decoupled Reusable Architecture rule (root `CLAUDE.md`). Updating the pin is a deliberate PR — no `git submodule update --remote` in any release script. The submodule is mirrored to GitHub + GitLab (2-upstream scope per 2026-04-30 spec deviation).

For a step-by-step recipe to add a third tracker, see `docs/sdk-developer-guide.md`. For the Challenge Test pack covering the SDK end-to-end on real devices, see `app/src/androidTest/kotlin/lava/app/challenges/`.

Phase 4–5 deliverables (SP-3a)

Phases 4 and 5 closed the multi-tracker SDK arc. Headline deliverables beyond what the SDK module map captures:

Phase 4 — Mirror health, cross-tracker fallback, settings UI

- `MirrorHealthCheckWorker` — `@HiltWorker` `PeriodicWorkRequest` scheduled every 15 minutes; HEAD-probes every registered mirror and writes the result to `tracker_mirror_health` (Room).
- `MirrorHealthRepository` — DAO wrapper exposing per-mirror health state observables.
- `UserMirrorRepository` — DAO wrapper for `tracker_mirror_user` (operator-added custom mirrors).
- `MirrorConfigLoader` — loads bundled defaults from `core/tracker/client/src/main/assets/mirrors.json`, validates per the schema in `docs/sdk-developer-guide.md` §7, merges with user customs.
- `CrossTrackerFallbackPolicy` — pure-function policy that decides whether to propose a cross-tracker fallback when the active tracker's mirrors all hit UNHEALTHY.
- `LavaTrackerSdk.search()` (and siblings) emit `SearchOutcome.CrossTrackerFallbackProposed(altTrackerId)` when the policy fires; **no silent fallback** — the SDK never re-routes without explicit user consent via the modal.

- `:feature:tracker_settings` — Compose UI with selector list, per-tracker mirror list section, health indicator, add-custom-mirror dialog. MVI via Orbit.
- `CrossTrackerFallbackModal` in `:feature:search_result` — the user prompt; accept re-issues the search on the alt tracker, dismiss shows explicit failure.
- Navigation entry: Settings → Trackers (wired from `:feature:menu`).

Phase 5 — Constitution + Challenges + tag gate

- Constitutional clauses 6.D (Behavioral Coverage Contract), 6.E (Capability Honesty), 6.F (Anti-Bluff Submodule Inheritance) added to root `CLAUDE.md`, cascaded to scoped clauses in `core/CLAUDE.md`, `feature/CLAUDE.md`, `lava-api-go/{CLAUDE,AGENTS}.md`, `submodules/tracker_sdk/{CLAUDE,CONSTITUTION,AGENTS}.md`, and root `AGENTS.md`.
- Seventh Law (Anti-Bluff Enforcement, all 7 clauses) added. Mechanical enforcement via `.githooks/pre-push`: Bluff-Audit commit-message stamp on every test commit, mock-the-SUT pattern rejection, hosted-CI config rejection.
- 8 Compose UI Challenge Tests (C1–C8) at `app/src/androidTest/kotlin/lava/app/challenges/`. Each carries a documented falsifiability rehearsal protocol in its KDoc.
- Local-Only CI/CD apparatus: `scripts/ci.sh` (single entry point; `--changed-only`, `--full`, `--smoke` modes), `scripts/check-fixture-freshness.sh`, `scripts/check-constitution.sh`, `scripts/bluff-hunt.sh` (Seventh Law clause 5 driver).
- `scripts/tag.sh` Android evidence-pack gate: refuses to tag without `.lava-ci-evidence/Lava-Android-<v>/` containing `ci.sh.json`, `challenges/C{1..8}.json` at status `VERIFIED`, `mirror-smoke/`, `bluff-audit/`, and `real-device-verification.md` at status `VERIFIED`.

Persistence schema (`AppDatabase` v6 → v7 migration)

```
// SP-3a Phase 4 added two tables:
@Entity(tableName = "tracker_mirror_health")
data class MirrorHealthEntity(
    @PrimaryKey val mirrorUrl: String,
    val trackerId: String,
    val state: String,                    // HEALTHY | DEGRADED |
        UNHEALTHY | UNKNOWN
    val lastProbedAt: Long,
    val lastHealthyAt: Long?,
    val consecutiveFailures: Int,
)

@Entity(tableName = "tracker_mirror_user")
data class UserMirrorEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val trackerId: String,
```

```
    val url: String,
    val priority: Int,
    val protocol: String,                // HTTPS | HTTP
)
```

The migration `MIGRATION_6_7` lives in `AppDatabase.kt` and creates both tables with the appropriate indices. Schema JSONs are checked into `core/database/schemas/lava.database.AppDatabase/{6,7}.json`.

Build System & Convention Plugins

All modules apply one or more custom convention plugins defined in `buildSrc`. The plugins are registered in `buildSrc/build.gradle.kts` and applied by ID.

Plugin ID	Applied To	What it does
`lava.android.application`	`:app`	Android app plugin, Firebase/Crashlytics, static analysis, Compose compiler
`lava.android.library`	Most `core:*` and `feature:*`	Android library + Kotlin Android + Spotless
`lava.android.library.compose`	Compose-enabled modules	Adds Compose BOM dependencies and compiler settings
`lava.android.feature`	All `feature:*`	Applies library + Hilt, wires standard core dependencies, Orbit, enables `-Xcontext-receivers`
`lava.android.hilt`	Android modules needing DI	Dagger Hilt plugin + KSP compilers
`lava.kotlin.library`	Pure Kotlin modules	

Plugin ID	Applied To	What it does
		`java-library` + Kotlin JVM + Spotless
`lava.kotlin.serialization`	Modules needing JSON serialization	Kotlin serialization plugin + `kotlinx-serialization-json`
`lava.kotlin.ksp`	Modules using KSP (e.g. Room)	KSP Gradle plugin
`lava.kotlin.tracker.module`	`:core:tracker:rutracker`, `:core:tracker:rutor`	Pure Kotlin module + serialization + Jsoup + OkHttp + coroutines + test deps for tracker plugins
`lava.ktor.application`	`:proxy`	Kotlin library + serialization + `application` plugin + Ktor plugin

Shared build constants live in `buildSrc/src/main/kotlin/lava/conventions/`:

- `AndroidCommon.kt` — `compileSdk = 35`, `minSdk = 21`, `targetSdk = 35`
- `KotlinAndroid.kt` — Java / Kotlin target `VERSION_17`
- `AndroidCompose.kt` — Compose compiler plugin + experimental opt-ins
- `StaticAnalysisConventionPlugin.kt` — Spotless configuration

Key build files

- `settings.gradle.kts` — Includes `:app`, `:api-app`, all `core:*` and `feature:*` modules, plus the `Tracker-SDK` composite build. (The legacy `:proxy` was removed 2026-05-06.)
- `gradle/libs.versions.toml` — Version catalog with libraries, plugins, and bundles (`coil`, `ktor`, `orbit`, `room`, `work`).
- `gradle.properties` — Standard Android properties (`android.useAndroidX=true`, `kotlin.code.style=official`, `android.nonFinalResIds=false`, etc.).

Build Commands

Because there is no root build script, you invoke tasks via the Gradle wrapper as usual. Common commands:

```
# Build the Android app (debug)
./gradlew :app:assembleDebug

# Build the Android app (release)
./gradlew :app:assembleRelease

# (The legacy `:proxy:buildFatJar` task was removed 2026-05-06 — the
#  backend is now lava-api-go; build it with `cd lava-api-go && make build`.)

# Run Spotless formatting/checks
./gradlew spotlessApply
./gradlew spotlessCheck

# Run all unit tests
./gradlew test

# Build all artifacts and copy to releases/
./build_and_release.sh
```

The `build_and_release.sh` script produces:

```
releases/
└── {version}/
    ├── android-debug/
    │   └── digital.vasic.lava.client-{version}-debug.apk
    ├── android-release/
    │   └── digital.vasic.lava.client-{version}-release.apk
    ├── proxy/
    │   └── digital.vasic.lava.api-{version}.jar
    └── api-go/
        └── lava-api-go (static binary) + Docker image tar
```

App build types

- **Debug** — `isMinifyEnabled = false`, signed with the custom debug keystore (`keystores/debug.keystore`), `applicationIdSuffix = ".dev"`.
- **Release** — `isRemoveUnusedCode = true`, `isRemoveUnusedResources = true`, `isOptimizeCode = true`, **but `isObfuscate = false`**. Uses ProGuard rules from `app/proguard-rules.pro` and is signed with the custom release keystore (`keystores/release.keystore`).

Go API service build commands

```
# Build the Go binaries (lava-api-go + healthprobe)
make build

# Run the Go test suite (unit + race detector)
make test

# Run the Go CI gate (tidy, codegen, vet, build, test, gosec,
        govulncheck)
make ci               # or: ./lava-api-go/scripts/ci.sh

# Regenerate OpenAPI types
make generate         # or: ./lava-api-go/scripts/generate.sh

# Build the Docker image
make image

# Run database migrations
make migrate-up       # requires LAVA_API_PG_URL
```

Code Style & Static Analysis

- **Spotless + ktlint** is the only enforced code-quality tool for Kotlin. It is configured programmatically in `buildSrc/src/main/kotlin/lava/conventions/StaticAnalysisConventionPlugin.kt`.
- There is **no Detekt** and **no Checkstyle**.
- **`.editorconfig`** exists at the project root and configures ktlint rules (e.g. allowing PascalCase for `@Composable` functions).
- Spotless targets:
  - All `**/*.kt` files (excluding `build/`)
  - All `*.gradle.kts` files
- Run `./gradlew spotlessApply` before committing.
- Kotlin code style is set to `official` in `gradle.properties`.
- For Go: `go vet ./...`, `gosec`, and `govulncheck` are run as part of `lava-api-go/scripts/ci.sh`.

Architecture & Module Organization

Clean architecture / layered modules

The project follows a **modular clean architecture** with strict dependency direction:

```
app
 └─> feature:*
     └─> core:domain
         └─> core:data
```

```
            └─> core:network:impl
            └─> core:database
        └─> core:auth:impl
        └─> core:work:impl
    └─> core:navigation, core:ui, core:designsystem
```

- **Pure Kotlin modules** (`core:models`, `core:common`, `core:auth:api`, `core:network:api`, `core:tracker:api`, `core:tracker:registry`, `core:tracker:mirror`, `core:tracker:rutracker`, `core:tracker:rutor`, `core:tracker:testing`, `core:work:api`) do not depend on the Android SDK.
- **API / Impl split** — Several core layers expose an `:api` module (interfaces + models) and an `:impl` module (Hilt-bound implementations). This keeps consumers decoupled from implementation details.

MVI with Orbit

Every feature module uses **Orbit MVI**:

- `XxxViewModel` — `@HiltViewModel`, implements `ContainerHost<State, SideEffect>`.
- `XxxState` — `sealed interface` describing UI states (`Loading`, `Loaded`, `Error`, `Empty`).
- `XxxAction` — `sealed interface` for user intents.
- `XxxSideEffect` — `sealed interface` for one-time events (navigation, toasts, etc.).

Example: `feature/forum/src/main/kotlin/lava/forum/ForumViewModel.kt`

UI & Navigation

- **100 % Jetpack Compose**. There are no XML layouts or Fragments in feature modules.
- **Custom design system** lives in `:core:designsystem` (`LavaTheme`, `AppBar`, custom text fields, etc.).
- **Custom navigation wrapper** in `:core:navigation` sits on top of Jetpack Navigation Compose.
  - Routes are built with a Kotlin DSL using **context receivers** (enabled in several modules).
  - Each feature exposes `addXxx()` and `openXxx()` extension functions.
  - Deep links for `rutracker.org/forum/viewtopic.php`, `viewforum.php`, and `tracker.php` are handled in `app/src/main/AndroidManifest.xml` and wired into the navigation graph.
- **TV support** — `TvActivity` extends `MainActivity` and changes `PlatformType` to `TV`. Leanback launcher intent is declared in the manifest. The app also declares `android.software.leanback` as not required and `android.hardware.touchscreen` as not required.

Anti-Bluff Testing Pact (Constitutional Law)

This project adheres to **Anti-Bluff Testing**. A "bluff test" is one that passes while the corresponding real feature is broken for end users. Bluff tests create false confidence and are strictly forbidden.

First Law — Tests Must Guarantee Real User-Visible Behavior

Every test MUST verify an outcome that matters to end users. A test that only asserts "function did not crash" or "mock was called" is a bluff test and must be rewritten.

Second Law — No Mocking of Internal Business Logic

- **ViewModel tests MUST use real UseCase implementations** wired to realistic fakes, never mocked use cases.
- **UseCase tests MUST use real Repository implementations** (or fakes that enforce identical invariants to the real database / network layer).
- Mocking is permitted ONLY at the outermost system boundaries (Android `NsdManager`, actual HTTP sockets, hardware).
- If a UseCase contains a bug, a ViewModel Integration Challenge Test wired to the real UseCase MUST fail.

Third Law — Fakes Must Be Behaviorally Equivalent

- A fake that is "simpler" than reality in a way that could hide a bug is a bluff fake.
- `TestEndpointsRepository` MUST reject duplicates (Room primary-key conflict) and seed defaults just like `EndpointsRepositoryImpl`.
- `TestLocalNetworkDiscoveryService` MUST simulate real NsdManager behaviors (e.g. `_lava._tcp.local.` service-type suffix).
- Every fake MUST document any behavioral simplifications that differ from production.

Fourth Law — Integration Challenge Tests

- Every feature MUST include at least one **Integration Challenge Test** that exercises the real implementation stack

end-to-end: ViewModel → UseCase → Repository → (Fake) Service.
- Challenge Tests use actual production classes at every layer; only external boundaries are faked.
- A passing Challenge Test MUST guarantee the feature works for a real user under the tested scenario.

Fifth Law — Regression Immunity

- Every production bug fix MUST be accompanied by a test that would have failed before the fix.
- If such a test cannot be written, the architecture is untestable and must be refactored before the fix is accepted.
- Code coverage numbers are meaningless if the tests are bluffs; behavioral guarantees are the only valid metric.

Sixth Law — Real User Verification (Anti-Pseudo-Test Rule)

A test that passes while the feature it covers is broken for end users is the most expensive kind of test in this codebase — it converts unknown breakage into believed safety. This has happened in this project before: tests and Integration Challenge Tests executed green while large parts of the product were unusable on a real device. That outcome is a constitutional failure, not a coverage failure, and it MUST NOT recur.

Every test added to this codebase from this point on MUST satisfy ALL of the following. Violation of any of them is a release blocker, irrespective of coverage metrics, CI status, reviewer sign-off, or schedule pressure.

1. **Same surfaces the user touches.** The test must traverse the production code path the user's action triggers, end to end, with no shortcut that bypasses real wiring. If the user's action is "open screen → tap button → see result", the test exercises the same screen, the same button handler, and the same result-rendering code — not a synthetic call into the ViewModel that skips the screen, and not a hand-rolled flow that skips the network/database boundary the real action crosses.

2. **Provably falsifiable on real defects.** Before merging, the author MUST run the test once with the underlying feature deliberately broken (throw inside the use case, return the wrong row from the repository, return the wrong status from the API) and confirm the test fails with a clear assertion message. The PR description MUST state which deliberate break was used and what failure the test produced. A test that cannot be made to fail by breaking the thing it claims to verify is a bluff test by definition.

3. **Primary assertion on user-visible state.** The chief failure signal of the test MUST be on something a real user could see or measure: rendered UI text, persisted database row, HTTP response body / status / header, file written to disk, packet on the wire. "Mock was invoked N times" is a permitted secondary assertion, never the primary one.

4. **Integration Challenge Tests are the load-bearing acceptance gate.** A green Challenge Test means a real user can complete the flow on a real device against real services — not "the wiring compiles". A feature for which a Challenge Test cannot be written is, by definition, not shippable. Refactor for testability or remove the feature.

5. **CI green is necessary, not sufficient.** "All tests pass" is not a release signal. Before any release tag is cut, a human (or a scripted black-box runner that drives the real UI / real HTTP API) MUST have used the feature on a real device or environment and observed the user-visible outcome described in the feature spec. Tag scripts (e.g. `scripts/tag.sh`) MUST NOT be run until that verification is documented.

6. **Inheritance.** This Sixth Law applies recursively to every submodule, every feature, and every new artifact added to the project (including the Go API service). Submodule constitutions MAY add stricter rules but MUST NOT relax this one.

Seventh Law — Tests MUST Confirm User-Reachable Functionality (Anti-Bluff Enforcement)

The Sixth Law states what tests must satisfy. The Seventh Law states how we enforce it mechanically — because the project has shipped passing-tests with broken-features at least once, and the operator's standing mandate (2026-04-30) is that this MUST NOT recur. The Seventh Law is the teeth.

Every commit that adds, modifies, or removes a test in this codebase, AND every commit that adds or modifies a user-facing feature, is bound by the seven clauses below. Violation is a release blocker — pre-push hooks reject the push, tag scripts refuse to operate, and reviewers MUST raise the violation as a blocking review comment. Authoritative text lives in `CLAUDE.md`; this AGENTS.md mirrors it so an agent reading either file sees the same rule set.

1. **Bluff Audit Stamp on every test commit.** Test-file diffs require a `Bluff-Audit:` block in the commit message body — name of the test, what was deliberately broken in the production code, the actual failure message observed, confirmation the mutation was reverted. Pre-push hook rejects test commits without the stamp.

2. **Real-Stack Verification Gate per feature.** Every user-visible feature requires a real-stack test: real third-party network for third-party services (`-PrealTrackers=true`), real Postgres for our own services (`-Pintegration=true`), real Compose UI on a real Android emulator/device for UI features (`-PdeviceTests=true` / `connectedAndroidTest`). Features without such a test are documented as "not user-reachable" until the gap closes.

3. **Pre-Tag Real-Device Attestation.** Every release tag requires `.lava-ci-evidence/<tag-name>/real-device-attestation.json` with: device model, Android version, app version, command-by-command checklist of executed user actions, ≥3 screenshots OR a video referenced by hash. `scripts/tag.sh` refuses without it. No exceptions.

4. **Forbidden Test Patterns.** Pre-push hooks reject diffs introducing: mocking the SUT, assertions that are only `verify { mock.foo() }`, `@Ignore`'d tests without a tracking issue, tests that build the SUT but never invoke its methods, acceptance gates whose chief assertion is BUILD SUCCESSFUL.

5. **Recurring Bluff Hunt.** Each phase (every 2-4 weeks of active work) ends only after a bluff hunt: 5 random test files, deliberately mutate the production code each one claims to cover, confirm each test fails. Output to `.lava-ci-evidence/bluff-hunt/<date>.json`.

6. **Bluff Discovery Protocol.** When a real user reports a bug whose tests are green, declare a Seventh Law incident. The fix commit MUST include a regression test that fails before the fix; the bluff that hid the bug MUST be diagnosed and recorded in `.lava-ci-evidence/sixth-law-incidents/<date>.json`; the Forbidden Test Patterns list MUST be updated.

7. **Inheritance and Propagation.** Applies recursively to every submodule, every feature, and every new artifact. Each submodule's `CLAUDE.md` MUST contain the verbatim Seventh Law or a link to this section. Non-compliant external submodules MUST be forked under `vasic-digital/` and brought into compliance before adoption.

Sixth Law extensions (lessons-learned addenda)

The clauses above are immutable. Below are addenda recorded after a real bug shipped green and we needed to prevent the *class* of bug, not just the specific instance. Authoritative copy lives in `CLAUDE.md`; this AGENTS.md mirrors the headers so an agent reading either file sees the same rule set.

6.A — Real-binary contract tests (added 2026-04-29)

Forensic anchor: `docker-compose.yml` invoked `healthprobe --http3 …` while the binary only registered `-url`/`-insecure`/`-timeout`; the probe exited 1 every 10 seconds for 569 consecutive runs while the API itself served 200. Compose validation was happy, every functional test was green, the orchestrator labelled the container "unhealthy" — and there was no test asserting the probe could even start.

Rule: every script/compose invocation of a binary we own MUST have a contract test that recovers the binary's flag set from its actual `Usage` output and asserts the script's flag set is a strict subset, with a falsifiability rehearsal sub-test that re-introduces a known-buggy flag and confirms the checker rejects it. Canonical implementation: `lava-api-go/tests/contract/healthcheck_contract_test.go`.

6.B — Container "Up" is not application-healthy (added 2026-04-29)

`docker ps` / `podman ps` `Up` only means PID 1 is alive; the application inside may be stuck or crash-looping internally (cf. 6.A). Tests asserting container state alone are bluffs by clauses 1 and 3. Use the same probe the orchestrator uses, or an end-to-end functional request at the user-visible surface.

6.C — Mirror-state mismatch checks before tagging (added 2026-04-29)

"All mirrors push succeeded" is one assertion; "all mirrors converge to the same SHA at HEAD" is stronger. `scripts/tag.sh` MUST verify post-push tip-SHA convergence across github/gitlab before reporting success, and SHOULD record per-mirror SHAs in the evidence file.

6.D — Behavioral Coverage Contract (added 2026-04-30, SP-3a)

Coverage is measured behaviorally, not lexically. Every public method of every interface added under `core/tracker/api/`, `submodules/tracker_sdk/api/`, or any future SDK contract module MUST have at least one real-stack test that traverses the same code path a user's action triggers. Line coverage is reported as a secondary metric. Uncovered lines after the behavioral pass are exempted only via an entry in the per-spec exemption ledger. Blanket coverage waivers are forbidden.

6.E — Capability Honesty (added 2026-04-30, SP-3a)

A `TrackerDescriptor` (or any future descriptor of a feature-bearing component) that declares a capability MUST cause `getFeature()` to return a non-null implementation for the corresponding feature interface. The historical "Not implemented" stub pattern is a constitutional violation. Capability declared ⇒ feature interface returned ⇒ at least one real-stack test exists for the capability.

6.F — Anti-Bluff Submodule Inheritance (added 2026-04-30, SP-3a)

Clauses 6.A through 6.E inherit recursively to every `vasic-digital` submodule mounted in this repository, to every future submodule, and to every code module added to a submodule. A submodule constitution MAY add stricter rules but MUST NOT relax 6.A–6.F. The Go API service (`lava-api-go/`) inherits 6.D and 6.E binding on its rutracker bridge work.

Local-Only CI/CD (Constitutional Constraint)

This project does NOT use, and MUST NOT add, GitHub Actions, GitLab pipelines, Bitbucket pipelines, CircleCI, Travis, Jenkins-as-a-service, Azure Pipelines, or any other hosted/remote CI/CD service. All build, test, lint, security-scan, mutation-test, load-test, image-build, and release-verification activity MUST run on developer machines or on a self-hosted local runner under the operator's direct control.

Why

1. **Sixth Law alignment.** Real-device, real-environment verification is load-bearing. Hosted CI runs in synthetic, ephemeral environments — green there proves the synthetic env, not the user's product on a real device.
2. **Hosted-CI green has historically produced bluff confidence in this project.** A remote dashboard saying "passing" is psychologically indistinguishable from "shipped and works", which is exactly the failure mode the Sixth Law was written to prevent.
3. **Vendor independence.** This project mirrors to two independent upstreams (GitHub and GitLab) intentionally. Coupling our quality gate to one vendor's pipeline product would re-introduce the single point of failure mirroring is meant to remove.

Mandatory consequences

- The `scripts/` directory (and any `Makefile`, task runner, or build tool introduced later) IS the CI/CD apparatus. Whatever runs in "release CI" MUST be the same script a developer runs locally — no parallel implementation.
- A single local entry point — `scripts/ci.sh` for Android and `lava-api-go/scripts/ci.sh` for Go — MUST invoke every quality gate appropriate to the changed surface.
- **Forbidden files.** No `.github/workflows/*`, `.gitlab-ci.yml`, `.circleci/config.yml`, `azure-pipelines.yml`, `bitbucket-pipelines.yml`, `Jenkinsfile` (for hosted Jenkins), or equivalent shall exist on any branch of any of the upstreams. A pre-push hook MUST reject pushes that introduce such files.
- **Pre-push gate.** A git pre-push hook installed under `.githooks/` (and enabled via `git config core.hooksPath .githooks`) MUST run the relevant subset of `scripts/ci.sh` before any push to any upstream. The hook MUST NOT be bypassable in routine work; `--no-verify` is reserved for documented emergencies and any such use MUST be noted in the next commit message.
- **Release tagging gate.** `scripts/tag.sh` MUST refuse to operate against any commit that has not been certified locally — i.e. the local CI gate must have been run successfully against the exact commit being tagged, and the result recorded in a tracked artifact (e.g. `.lava-ci-evidence/`).
- **No "it passes on CI" handwave.** A failure that reproduces locally on the developer's machine takes precedence over any other signal. Conversely, a developer who claims "it works for me" without having run the local CI gate has not actually verified anything per the Sixth Law.

What this rule does NOT forbid

- Running tests, linters, or scanners *on the developer's machine* via any tool (this is the whole point).
- Running tests inside containers locally (this is the whole point).
- Running a self-hosted runner (a real machine the operator controls, invoking the same `scripts/ci.sh`) — that is local CI by another name.
- Per-upstream branch-protection rules that simply *require* status checks to be green; we just don't satisfy those checks via hosted runners.

Inheritance

Every submodule and every new artifact added to the project (including the Go API service) inherits this rule. Submodule constitutions MAY add stricter local-CI requirements but MUST NOT relax this one or introduce hosted-CI configuration.

Decoupled Reusable Architecture (Constitutional Constraint)

EVERY component that has a non-Lava-specific use case MUST live in a `vasic-digital` submodule, not in this repo. The boundary is enumerated per-component in the design doc for the sub-project that introduces the component (SP-2 onward). Code that ends up in this repo MUST be either:

- (a) **Lava domain logic** — the 13 rutracker routes, the rutracker HTML parsers, the Compose UI for Lava screens, the `Endpoint` sealed-interface, app/manifest entries, etc.; OR
- (b) **Thin glue** tying `vasic-digital` submodules together for Lava's specific needs.

Why

1. **Sixth Law alignment.** "It works in Lava" must mean the same thing as "it works in the next vasic-digital project that uses this submodule" — otherwise we are silently coupling generic capability to one product, which is the same class of failure as silently disabling rate-limiting (Sixth Law clause 5 territory).
2. **Bluff prevention.** Code copy-pasted between projects is the highest-bandwidth bluff vector: behaviour drifts silently, fixes don't propagate, "we have an mDNS implementation" turns out to mean four divergent implementations with one bug each.
3. **Operator economics.** The `vasic-digital` org is owned end-to-end. New repos under it cost zero (we own the org, we have CLI access on GitHub and GitLab). Reusing existing submodules costs zero pinning effort. Re-implementing per-project is the only expensive option.

Mandatory consequences

- **Submodule pins are explicit and frozen by default.** A pinned submodule does NOT auto-fetch latest; we are not obligated to track upstream movement. Frozen forever is acceptable. Updating the pin is a deliberate PR.
- **New non-Lava-specific code added to this repo without a documented "why not a vasic-digital submodule" decision is rejected.** The decision MUST appear either in the relevant design doc or in the PR description.

- **Generic functionality is contributed UPSTREAM first.** Any component that another `vasic-digital` project would conceivably want goes to the appropriate submodule (or a new `vasic-digital/<name>` repo). Lava then pins to the new hash. Order matters: upstream first, Lava pin second.
- **Every `vasic-digital` submodule we own inherits the Sixth Law and the Local-Only CI/CD rule transitively.** Adopting an externally maintained submodule that violates either is forbidden — fork it under `vasic-digital/` and adopt the fork.
- **Submodule fetch/pull is an EXPLICIT operator action, never automatic.** No git hooks that silently update pins, no `git submodule update --remote` in any release script. The pin is the contract; changing the contract is a code review event.
- **Mirror policy applies recursively.** Every `vasic-digital` submodule we own MUST be mirrored to the same set of upstreams Lava itself mirrors to (GitHub + GitLab), unless explicitly waived for that submodule with a documented reason.

What this rule does NOT forbid

- Lava-domain code in this repo. The 13 rutracker routes, the rutracker scrapers, the Compose UI, the `Endpoint` model, the `:proxy` Ktor server, the `app/` Android module — all stay here.
- Thin Lava-specific glue files in this repo that compose `vasic-digital` primitives.
- Deferring extraction of a borderline piece during a single PR; the deferral must be tracked as a TODO with a target sub-project for extraction.

Inheritance

This Decoupled Reusable Architecture rule applies recursively to every `vasic-digital` submodule we own. A submodule constitution MAY add stricter rules (e.g. "no external dependencies") but MUST NOT relax this one — meaning, `vasic-digital` submodules themselves MUST extract their own reusable parts to deeper submodules rather than copy-paste between siblings.

Testing Strategy

Post-SP-3a (1.2.0) the repository carries:

- **~77 `*Test.kt` files** across `core/*` and `feature/*`, with the `:core:tracker:*` family alone contributing the majority.
- **8 Compose UI Challenge Tests** at `app/src/androidTest/kotlin/lava/app/challenges/` (C1–C8). Each is the load-bearing acceptance gate for one user-visible scenario per Sixth Law clause 4.

- **1 real-tracker integration test** at `core/tracker/rutor/src/integrationTest/kotlin/lava/tracker/rutor/RealRuTorIntegrationTest.kt`, gated by `-PrealTrackers=true`.
- A shared test-utilities module — `:core:testing` — providing `MainDispatcherRule`, fake repositories/services, `TestDispatchers`, and the Compose UI test harness.
- A separate tracker-test scaffolding module — `:core:tracker:testing` — providing `FakeTrackerClient`, builders, and `LavaFixtureLoader`.

Test discipline (Anti-Bluff Pact):

- Use **JUnit 4** for unit tests (to match the existing `MainDispatcherRule`).
- Place unit tests in `src/test/kotlin/`, instrumented tests in `src/androidTest/kotlin/`, real-tracker integration tests in `src/integrationTest/kotlin/` (only `:core:tracker:rutor` has this source set today; `:core:tracker:rutracker` adds it as needed).
- Every feature MUST include at least one **Integration Challenge Test** using real UseCase and Repository implementations. Mocking ViewModel dependencies is a Second-Law violation.
- Every test commit MUST carry a **Bluff-Audit** stamp (Seventh Law clause 1, pre-push hook enforced).
- Mocking the System Under Test (`mockk<XxxClass>(...)` inside `XxxClassTest.kt`) is **forbidden** and pre-push-rejected (Seventh Law clause 4).

Test runner cheatsheet

All commands assume working directory at the repo root.

Goal	Command
Default unit tests, all modules	`./gradlew test`
Unit tests, single tracker module	`./gradlew :core:tracker:rutor:test`
Unit tests, single test class	`./gradlew :core:tracker:rutor:test --tests "RuTorSearchParserTest"`
Real-tracker integration test (RuTor only)	`./gradlew :core:tracker:rutor:integrationTest -PrealTrackers=true`
Compose UI Challenge Tests (all 8)	`./gradlew :app:connectedDebugAndroidTest` *(requires connected device)*

Goal	Command
Compose UI single Challenge Test	`./gradlew :app:connectedDebugAndroidTest --tests "lava.app.challenges.Challenge01*"`
Local CI gate, changed-only (matches pre-push hook)	`./scripts/ci.sh --changed-only`
Local CI gate, full (matches `scripts/tag.sh` gate)	`./scripts/ci.sh --full`
Local CI gate, smoke (cheapest, sanity check)	`./scripts/ci.sh --smoke`
Recurring bluff hunt (Seventh Law clause 5, end-of-phase)	`./scripts/bluff-hunt.sh`
Constitution checker (capability honesty + scoped clauses)	`./scripts/check-constitution.sh`
Fixture freshness (warn at >30d, block at >60d)	`./scripts/check-fixture-freshness.sh`
Tracker-SDK submodule mirror sync (operator-only)	`./scripts/sync-tracker-sdk-mirrors.sh --check`
Spotless	`./gradlew spotlessApply` / `./gradlew spotlessCheck`
Single ViewModel test	`./gradlew :feature:tracker_settings:test --tests "TrackerSettingsViewModelTest"`
Go API unit tests	`cd lava-api-go && go test -race -count=1 ./...`
Go API CI gate	`./lava-api-go/scripts/ci.sh`
Go API e2e tests	`cd lava-api-go && go test -race -count=1 -v ./tests/e2e/...`
Go API contract tests	`cd lava-api-go && go test -race -count=1 -v ./tests/contract/...`

Gradle `--no-daemon` recommendation. For long-running builds (the Android `:app:assembleDebug` end-to-end takes minutes on a cold cache), prefer `--no-daemon` to avoid the Gradle daemon holding locks across a host suspend (Host Machine Stability Directive).

`-Pintegration=true` and `-PdeviceTests=true`. Reserved for future extensions of the real-stack gate (lava-api-go integration suites and Android device tests respectively). 1.2.0 ships only `-PrealTrackers=true` for the RuTor integration set; the others are placeholders documented in the Seventh Law clause 2.

Deployment

Android App

The app is distributed via Google Play, GitHub Releases, and RuStore (per `README.md`). The release-tagging mechanics are local-only per the Local-Only CI/CD constitutional rule:

- `./scripts/ci.sh --full` is the local quality gate. The same script runs in pre-push (`--changed-only` mode) and at tag time (`--full` mode).
- `./scripts/tag.sh --app android` cuts the `Lava-Android-<vname>-<vcode>` tag, pushes to both upstreams (github + gitlab), and verifies per-mirror SHA convergence per clause 6.C. It refuses to operate without a populated `.lava-ci-evidence/Lava-Android-<vname>-<vcode>/` evidence pack (Seventh Law clause 3 mechanical gate).

There are NO `.github/workflows/`, `.gitlab-ci.yml`, `.circleci/`, or any other hosted-CI configuration files anywhere in the tree. The pre-push hook at `.githooks/pre-push` rejects pushes that introduce them (Local-Only CI/CD rule).

Go API Service

The Go API is the primary backend. It is built as a static binary and a distroless Docker image.

1. **Build the binaries:**

   ```
   cd lava-api-go && make build
   ```

   Output: `lava-api-go/bin/lava-api-go` and `lava-api-go/bin/healthprobe`

2. **Build the Docker image:**

   ```
   cd lava-api-go && make image
   ```

Image: `lava-api-go:dev`

3. **Dockerfile** (`lava-api-go/docker/Dockerfile`):
   - Multi-stage build: `golang:1.25-alpine` → `gcr.io/distroless/static-debian12:nonroot`
   - Exposes `8443/udp` (HTTP/3) and `8443/tcp` (HTTP/2)
   - `HEALTHCHECK CMD ["/usr/local/bin/healthprobe"]` (JSON-array form for distroless compat)
   - Entrypoint: `/usr/local/bin/lava-api-go`
4. **Orchestration:** `docker-compose.yml` at the project root defines services for Postgres, migrations, lava-api-go, and an observability stack (Prometheus, Loki, Grafana, Tempo). Use `./start.sh` and `./stop.sh` to manage the stack. (The legacy proxy service was removed 2026-05-06.)

Proxy Server (Legacy) — REMOVED 2026-05-06

The legacy Ktor/Netty proxy was deleted in the SP-2 migration; `lava-api-go` is the sole backend. There is no `proxy/` directory, no `:proxy:buildFatJar` task, and `build_and_push_docker_image.sh` now builds the lava-api-go image. To build/deploy the backend, see the lava-api-go deployment section and `cd lava-api-go && make build && make image`.

Security Considerations

- **Go API auth** — The Go API uses passthrough auth middleware that forwards the `Auth-Token` header as a cookie to upstream rutracker.org. (The legacy `:proxy`'s equivalent auth was removed with the module on 2026-05-06.)
- **Encrypted preferences** — `core:preferences` uses `androidx.security:security-crypto-ktx` (`1.1.0-alpha03`) to store credentials and settings.
- **Cleartext traffic** — `android:usesCleartextTraffic="true"` is enabled in the app manifest. Be cautious when changing this.
- **Signing configuration** — Both debug and release builds use dedicated keystores located under `keystores/`. Keystore paths and passwords are loaded from a `.env` file via `KEYSTORE_ROOT_DIR` and `KEYSTORE_PASSWORD`. See `.env.example` for the required variables. Never commit the `.env` file or the `keystores/` directory.
- **Credential Security (Constitutional clause 6.H)** — `.env`, `.env.*`, `.env.local`, `keystores/`, and `app/google-services.json` are inviolable. They MUST be gitignored. No credential string shall ever appear in source code. The pre-

push hook rejects any diff that introduces credential patterns. Accidental commit is a security incident requiring upstream purge and credential rotation.

- **Provider Operational Verification (Constitutional clause 6.G)** — Every `TrackerDescriptor` shipped to end users MUST have at least one passing Challenge Test exercising its primary user-visible flow on a real Android device or in the project's emulator container. The descriptor's `verified: Boolean` flag is the mechanical gate; the user-facing provider list (login, settings, credentials screens) MUST filter on `verified == true`. A provider that appears selectable but cannot complete its primary flow is a constitutional violation, irrespective of unit-test coverage. Forensic anchor: 2026-05-04 Internet Archive stuck-on-loading bug — Challenge Tests C1–C8 were green while archive.org users could not progress past the Continue tap because the test descriptors used `AuthType.FORM_LOGIN` and never exercised the `AuthType.NONE` code path. Pinned by `core/tracker/client/src/test/kotlin/lava/tracker/client/ProviderVerifiedContractTest.kt` — flipping `verified=true` without the matching Challenge Test fails CI immediately.

- **Multi-Emulator Container Matrix (Constitutional clause 6.I)** — Real-device verification (per 6.G clause 5 / Sixth Law clause 5 / Seventh Law clause 3) is satisfied ONLY by the project's container-bound multi-emulator matrix: minimum API 28, 30, 34, latest stable; minimum phone + tablet + (where the feature touches `TvActivity` or the leanback manifest entries) TV; per-AVD attestation rows in `.lava-ci-evidence/<tag>/real-device-verification.md`; cold-boot for the gate run; falsifiability rehearsal per Android-version-class. A single passing emulator (or device) is NOT the gate — the matrix is. `scripts/tag.sh` MUST refuse to operate on a commit whose evidence file lacks the minimum-coverage rows. Snapshot reuse during dev is fine; for the gate it is forbidden.

- **Anti-Bluff Functional Reality Mandate (Constitutional clause 6.J)** — Every test, every Challenge Test, and every CI gate added to or maintained in this codebase MUST do exactly one job: confirm the feature it claims to cover actually works for an end user, end-to-end, on the gating matrix (clause 6.I). CI green is necessary, NEVER sufficient. Any agent or contributor may invoke clause 6.J to remove a test demonstrably bluff (passes against deliberately-broken production code) — the bluff classification + broken-mutation evidence go to `.lava-ci-evidence/sixth-law-incidents/<date>.json`. Adding a Challenge Test the author cannot execute against the gating matrix is itself a bluff. `@Ignore` without an open issue is a 6.J violation by construction. Tests must guarantee the product works — anything else is theatre.

- **Builds-Inside-Containers Mandate (Constitutional clause 6.K)** — Every release-artifact build (`:app:assembleDebug`, `:app:assembleRelease`, `:proxy:buildFatJar`, `lava-api-go` static binary, OCI image builds) MUST run inside the project's container-bound build path, anchored on `vasic-digital/Containers`'s build orchestration (`cmd/distributed-build` + `pkg/distribution` + `pkg/runtime`). Local incremental dev builds on the host are permitted for iteration; the constitutional gate, the release-artifact build, and the build whose output goes through the emulator matrix (clause 6.I) MUST go through Containers. The accompanying `6.K-debt` entry tracks the package additions (`submodules/containers/pkg/emulator/`, `submodules/containers/pkg/vm/`) that are owed; until that debt is closed, Lava-side `docker-compose.test.yml`, `docker/emulator/Dockerfile`, and `scripts/run-emulator-tests.sh` are the transitional thin glue. An artifact built outside the container path and tested inside the gating emulator path is constitutionally suspect — green tests against an artifact the gate did not build are a bluff vector by construction.

- **Anti-Bluff Functional Reality Mandate, Operator's Standing Order (Constitutional clause 6.L)** — Restated verbatim from clause 6.J because the operator has now invoked this mandate **TWENTY-FIVE TIMES** across multiple working days; the repetition itself is the forensic record. The 13th invocation (2026-05-05 23:51, after a real tester reported the Trackers-from-Settings crash): "Opening Trackers from Settings crashes the app. See Crashlytics for stacktraces. Create proper tests to validate and verify the fix! ... execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!" The crash root cause was a textbook nested-scroll antipattern (LazyColumn inside Column(verticalScroll)) that no existing test caught — confirming yet again that CI green is necessary, never sufficient. (See §6.Q.) The 14th invocation (2026-05-06) birthed §6.R No-Hardcoding Mandate. The 15th invocation (2026-05-06) birthed §6.S Continuation Document Maintenance Mandate. The 16th invocation (2026-05-12, after the C03+CF commits `4d27c07+f7d0a62` landed on master): "Make sure that all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!" The 17th invocation (2026-05-12 later, after the anti-bluff audit response `4b0dd55` landed) used the same verbatim wording and demands that the rule live in every submodule's CONSTITUTION.md +

AGENTS.md too, not just CLAUDE.md — the 17th audit verified all 16 submodule + lava-api-go CONSTITUTION/AGENTS files already carry §6.L inheritance.

- **Compose Layout Antipattern Guard (Constitutional clause 6.Q, added 2026-05-05)** — Forbids nesting a vertically-scrolling lazy layout (LazyColumn, LazyVerticalGrid, LazyVerticalStaggeredGrid) inside a parent that gives unbounded vertical space (verticalScroll, unbounded wrapContentHeight, LinearLayout-with-weight wrapper). Equivalent rule horizontally. Per-feature structural tests + Compose UI Challenge Tests on the §6.I matrix are the load-bearing acceptance gates. Forensic anchor: the 2026-05-05 Trackers-from-Settings crash (closure log at `.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md`). Inherits recursively to every submodule. Pattern guard at `feature/tracker_settings/src/test/.../TrackerSelectorListLazyColumnRegressionTest.kt`. Every test, every Challenge Test, every CI gate has exactly one job: confirm the feature works for a real user end-to-end on the gating matrix. CI green is necessary, never sufficient. Tests must guarantee the product works — anything else is theatre. If you find yourself rationalizing a "small exception" — STOP. There are no small exceptions. The Internet Archive stuck-on-loading bug, the broken post-login navigation, the credential leak in C2, the bluffed C1-C8, the 2 first-distribute Crashlytics crashes — these are what "small exceptions" produce. Submodule constitutions MAY paste this clause verbatim; they MUST NOT abbreviate it.

6.R — No-Hardcoding Mandate (added 2026-05-06, FOURTEENTH §6.L invocation)

Forensic anchor: 2026-05-06 operator directive during Phase 1 brainstorm: "Pay attention that we MUST NOT hardcode anything ever!" — restating the spirit of §6.J for an entire class of bluffs (literal values that drift silently from their intended source-of-truth).

Rule. No connection address, port, header field name, credential, key, salt, secret, schedule, algorithm parameter, or domain literal shall appear as a string/int constant in tracked source code (`.kt`, `.java`, `.go`, `.gradle`, `.kts`, `.xml`, `.yaml`, `.yml`, `.json`, `.sh`). Every such value MUST come from a config source: `.env` (gitignored), generated config class (build-time codegen reading `.env`), runtime env var, or mounted file.

The placeholder file `.env.example` (committed) carries dummy values for every variable so a developer cloning the repo knows what to set.

`scripts/check-constitution.sh` MUST grep tracked files for forbidden literal patterns:

- Any IPv4 address outside `.env.example` and incident docs
- The header name from `.env.example`'s `LAVA_AUTH_FIELD_NAME`
- Any 36-char UUID outside `.env.example`
- Hardcoded `host:port` pairs in HTTP/HTTPS URLs

Pre-push rejects on match. Bluff-Audit stamp required on any commit that adds new config-driven values, demonstrating the no-hardcoding contract test fails when a literal is reintroduced.

Exemptions (test fixtures, incident docs, design specs):

- `.env.example` — by definition carries placeholders
- `.lava-ci-evidence/sixth-law-incidents/*.json` — forensic anchors quoting historical literals
- `docs/superpowers/specs/*.md`, `docs/superpowers/plans/*.md` — design docs and implementation plans may show example values for clarity (placeholders preferred but examples permitted)
- `*_test.go`, `*Test.kt`, `*Tests.kt`, `*Test.java` — test fixtures may use synthetic literals, MUST NOT use real production values

Enforcement status (2026-05-06): UUID literals are enforced today by `scripts/check-constitution.sh` (delegating to `scripts/scan-no-hardcoded-uuid.sh`). IPv4, host:port, schedule, and algorithm-parameter literals are tracked for future phases per the staged-enforcement principle (rule first, mechanical gate as the project ships its scopes). Open a forensic-anchor entry in `.lava-ci-evidence/sixth-law-incidents/` if a literal of any forbidden class ships in production.

Inheritance: applies recursively to every submodule and every new artifact. Submodule constitutions MAY add stricter rules but MUST NOT relax this clause.

6.S — Continuation Document Maintenance Mandate (added 2026-05-06, FIFTEENTH §6.L invocation)

Forensic anchor: 2026-05-06 operator directive: "during any work we perform, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document MUST BE maintained and it MUST NOT BE out of sync with current work we are doing! If for any reson we stop our work, we MUST BE able to continue any time, with current work, exactly where we have left of and from any CLI agent or any LLM model we chose! Nothing can be broken or faulty in maintained Continuation document!"

Rule. The file `docs/CONTINUATION.md` is the single-file source-of-truth handoff document for resuming the project's work across any CLI session. It MUST be maintained continuously and MUST NOT drift out of sync with the actual repository state. Every commit that changes any of: (a) phase status, (b) new spec/plan, (c) submodule pin, (d) release artifact, (e) known issue discovered or resolved, (f) operator scope directive, (g) user-visible bug fix — MUST update `docs/CONTINUATION.md` in the SAME COMMIT.

The §0 "Last updated" line MUST be the date of the most recent CONTINUATION-touching commit. A stale value is a §6.S violation.

`scripts/check-constitution.sh` enforces (1) `docs/CONTINUATION.md` exists, (2) contains §0 "Last updated", (3) contains §7 RESUME PROMPT, (4) §6.S present in `CLAUDE.md`, (5) §6.S inheritance reference in every `submodules/*/CLAUDE.md` + `lava-api-go/CLAUDE.md`. Pre-push rejects on any failure.

Inheritance: applies recursively. Submodule constitutions MAY add stricter rules (e.g., per-submodule CONTINUATION files) but MUST NOT relax this clause.

6.T — Universal Quality Constraints (added 2026-05-06 from ../HelixCode constitution mining)

Four constitutional anchors imported verbatim from the HelixCode project's CONSTITUTION.md:

- **§6.T.1 Reproduction-Before-Fix (HelixCode CONST-014).** Every defect MUST be reproduced by a Challenge / regression test BEFORE the fix lands. Sequence: write test → confirm fail → fix → confirm pass → commit both with the failure message in a Bluff-Audit stamp.
- **§6.T.2 Resource Limits for Tests & Challenges (HelixCode CONST-011).** Full-suite test runs MUST cap host-resource usage (~30-40%) via `GOMAXPROCS=2 nice -n 19 ionice -c 3 -p 1` for `go test`, `--cpus`/`--memory` for container test runs, `--max-workers=2` for Gradle full-tree runs.
- **§6.T.3 No-Force-Push (HelixCode §12.2 / CONST-043).** No force-push, history rewrite, branch-deletion, or hook-bypass without explicit per-operation operator approval. The 4-mirror `-s ours` reconciliation pattern is the prescribed alternative.
- **§6.T.4 Bugfix Documentation (HelixCode CONST-012).** All bug fixes documented in `docs/issues/fixed/BUGFIXES.md` with root cause, files, fix description, verification test link, fix commit SHA. §6.O extends this for Crashlytics issues; §6.T.4 covers the rest.

Inheritance: applies recursively. Submodules MAY add stricter rules but MUST NOT relax.

- **Crashlytics-Resolved Issue Coverage Mandate (Constitutional clause 6.O, added 2026-05-05)** — Every Crashlytics-recorded issue (fatal OR non-fatal) closed/resolved by any commit MUST gain (a) a validation test in the language of the crashing surface that reproduces the conditions, (b) a Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/` (client) or `tests/e2e/` (server) that drives the same user-facing path, and (c) a closure log at `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` recording the issue ID, root-cause analysis, fix commit SHA, and links to the tests. `scripts/tag.sh` MUST refuse release tags whose CHANGELOG mentions Crashlytics fixes without matching closure logs. Marking a Crashlytics issue "closed" in the Console requires the test coverage to land first — never close-mark before the regression-immunity tests exist. Inherits recursively to every submodule.

- **Distribution Versioning + Changelog Mandate (Constitutional clause 6.P, added 2026-05-05, TWELFTH §6.L invocation)** — Every distribute action (Firebase App Distribution, container registry pushes, releases/ snapshots, scripts/tag.sh) MUST: (1) carry a strictly increasing versionCode (no re-distribution of already-published codes); (2) include a CHANGELOG entry — canonical file `CHANGELOG.md` at repo root + per-version snapshot at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md`; (3) inject the changelog into the App Distribution release-notes via `--release-notes`. `scripts/firebase-distribute.sh` REFUSES to operate when current versionCode ≤ last-distributed versionCode for the channel, OR when CHANGELOG.md lacks an entry for the current version, OR when the per-version snapshot file is missing. `scripts/tag.sh` enforces the same gates pre-tag. Re-distributing the same versionCode is forbidden across distribute sessions; idempotent retry within a single session is permitted. ServiceAdvertisement.API_VERSION MUST track proxy apiVersionName (per the §6.A real-binary contract test). Inherits recursively to every submodule + every new artifact.

- **Host-Stability Forensic Discipline (Constitutional clause 6.M)** — Every perceived-instability event during a session — whether a real host suspend, a real sign-out, OR an operator-perceived freeze without forensic evidence — MUST be classified into Class I (verifiable host event), Class II (resource pressure), or Class III (operator-perceived without forensic evidence) AND audited via the 7-step forensic protocol (uptime+who, journalctl logind events, kernel critical events, free -h, df -h, forbidden-command grep, container state inventory). Findings recorded under `.lava-ci-evidence/sixth-`

`law-incidents/<date>-<slug>.json`. Container-runtime safety analysis (recorded once in root §6.M, referenced forever): rootless Podman has NO host-level power-management privileges; rootful Docker is not installed on the operator's primary host. Container operations cannot cause Class I host events on the audited host configuration. The Forbidden Command List remains the prohibition; clause 6.M is the discipline for classifying and recording perceived events that may or may not have actually occurred. A perceived-instability event without an audit record is itself a Seventh Law violation. Forensic anchor: 2026-04-28 Class I poweroff (`docs/INCIDENT_2026-04-28-HOST-POWEROFF.md`) + 2026-05-04 Class III perceived-instability (`.lava-ci-evidence/sixth-law-incidents/2026-05-04-perceived-host-instability.json`).

- **Bluff-Hunt Cadence Tightening + Production Code Coverage (Constitutional clause 6.N)** — Beyond the Seventh Law clause 5 baseline (5 random `*Test.kt` files every 2-4 weeks), three additional triggers fire in-cycle: per operator anti-bluff-mandate invocation (lighter scope after first/day), per matrix-runner / gate change (pre-push enforced — owed via §6.N-debt), per phase-gating attestation file added (pre-push enforced — owed via §6.N-debt). Bluff hunts MUST also sample production code: 2 files per phase from gate-shaping code (`scripts/tag.sh` helpers, `scripts/check-constitution.sh`, `submodules/containers/pkg/emulator/`, etc.) plus 0-2 from broader CI-touched code. Forensic anchor: 2026-05-05 ultrathink-driven discovery of the 7-day-old `pkg/emulator/Boot()` port-collision bluff — invisible to all existing test-only bluff hunts. Pre-push hook implementation owed via the Group A-prime spec (next brainstorming target after Group A lands).

- **No sudo/su ever (Constitutional clause 6.U)** — Every use of `sudo` or `su` is strictly forbidden. Operations requiring elevated privileges MUST use container-based solutions from the `vasic-digital/Containers` submodule or be provided by local project/Submodule dependencies that build automatically. Never modify a suggestion to "just use sudo" — find a user-level alternative. The pre-push hook rejects files containing `sudo` or `su` patterns. Propagated recursively to all submodules.

- **Container Emulators mandate (Constitutional clause 6.V)** — Every Android emulator instance for Challenge Tests / UI verification MUST run inside a container managed by the `vasic-digital/Containers` submodule. Rootless Podman/Docker only. All tests execute inside containers. The §6.I matrix (API 28/30/34/latest, phone/tablet/TV) runs inside container-bound emulators. Propagated recursively.

- **GitHub + GitLab only remotes (Constitutional clause 6.W)** — Only GitHub (`vasic-digital/*`, `HelixDevelopment/*`) and GitLab (`vasic-digital/*`, `HelixDevelopment/*`) are permitted as

remotes. GitFlic, GitVerse, and all other providers are forbidden. The 4-mirror model is replaced by 2-mirror (GitHub + GitLab). All scripts, verification, and attestation procedures check only 2 mirrors. Propagated recursively.

- **ProGuard** — `isObfuscate = false` in release. The ProGuard rules in `app/proguard-rules.pro` keep network DTOs (`lava.network.dto.**`) and Tink classes.
- **Deep links** — The app handles `rutracker.org` deep links. Any Intent processing should validate URLs to avoid injection.
- **Go API TLS** — The Go API requires TLS 1.3. Self-signed dev certs are generated by `lava-api-go/scripts/gen-cert.sh`. Production requires valid certs.
- **Go API rate limiting** — Per-IP and per-route rate limiting is enforced via the `digital.vasic.ratelimiter` submodule.

Useful Notes for Agents

- **No root `build.gradle.kts`** — Do not create one. Use or extend the convention plugins in `buildSrc` instead.
- **Adding a new module?** Register it in `settings.gradle.kts` and apply the appropriate convention plugin (`lava.android.feature`, `lava.kotlin.library`, `lava.kotlin.tracker.module`, etc.).
- **Adding a dependency?** Prefer adding it to `gradle/libs.versions.toml` first, then referencing it via `libs.findLibrary(...)` or `libs.findBundle(...)` in the convention plugin or module `build.gradle.kts`.
- **Compose compiler** is managed via the Kotlin Compose compiler plugin (BOM `2025.06.01`). Do not add the old `composeOptions` block.
- **Context receivers** are enabled in several modules (`-Xcontext-receivers`). If you see `context(NavigationGraphBuilder)` DSL calls, that is why.
- **Firebase/Google Services** — The app plugin expects `google-services.json` in the `app/` directory. It is gitignored (`app/.gitignore`).
- **Avoid XML** — The project is fully Compose. Do not add XML layouts or Fragment-based screens.
- **Orbit MVI** — When modifying a feature, keep the `ViewModel` / `State` / `Action` / `SideEffect` pattern consistent with neighboring features.
- **Go API spec-first** — `lava-api-go/api/openapi.yaml` is the source of truth. Go types are generated via `oapi-codegen`. The generated code strips `,omitempty` from JSON tags to match Ktor's kotlinx-serialization wire shape.
- **Go API codegen invariant** — `scripts/generate.sh` regenerates `internal/gen/` from the OpenAPI spec. CI enforces that "regenerate produces empty diff."

- **`BUILDAH_FORMAT=docker`** — Enforced in `start.sh`, `stop.sh`, and `build_and_release.sh` to preserve `HEALTHCHECK` directives in Podman's default OCI format.

Host Machine Stability Directive (Critical Constraint)

IT IS FORBIDDEN to directly or indirectly cause the host machine to:

- Suspend, hibernate, or enter standby mode
- Sign out the currently logged-in user
- Terminate the user session or running processes
- Trigger any power-management event that interrupts active work

Why This Matters

AI agents may run long-duration tasks (builds, tests, container operations). If the host suspends or the user is signed out, all progress is lost, builds fail, and the development session is corrupted. This has happened before and must never happen again.

What Agents Must NOT Do — Explicit Forbidden Command List

The following invocations are **categorically forbidden** in any committed script, any subagent's planned action, or any agent's tool call:

```
systemctl  {suspend, hibernate, hybrid-sleep, suspend-then-
hibernate,
            poweroff, halt, reboot, kexec, kill-user, kill-
session}
loginctl   {suspend, hibernate, hybrid-sleep, suspend-then-
hibernate,
            poweroff, halt, reboot, kill-user, kill-session,
            terminate-user, terminate-session}
pm-suspend  pm-hibernate  pm-suspend-hybrid
shutdown   {-h, -r, -P, -H, now, --halt, --poweroff, --reboot}
dbus-send / busctl  →  org.freedesktop.login1.Manager.{Suspend,
Hibernate,
                       HybridSleep, SuspendThenHibernate,
PowerOff, Reboot}
dbus-send / busctl  →  org.freedesktop.UPower.{Suspend, Hibernate,
HybridSleep}
gsettings set       →  *.power.sleep-inactive-{ac,battery}-type
```

```
set to anything
                        except 'nothing' or 'blank'
gsettings set       →   *.power.power-button-action  set to
anything except
                        'nothing' or 'interactive'
```

Additional rules:

- Never modify power-management settings (sleep timers, lid-close behavior, screensaver activation)
- Never trigger a full-screen exclusive mode that might interfere with session keep-alive
- Never run commands that could exhaust system RAM and trigger an OOM kill of the desktop session
- Never execute `killall`, `pkill`, or mass-process-termination targeting session processes

Forensic record: incident 2026-04-28 18:37 (host poweroff)

See `docs/INCIDENT_2026-04-28-HOST-POWEROFF.md` for the full investigation. A user-space-initiated graceful poweroff via GNOME Shell occurred at 18:37:14 during SP-2 implementation. **Not a suspend, not a hibernate, not an OOM kill, not a kernel panic.** Root cause was external to this codebase (operator GNOME power-button click, hardware power-button, or out-of-scope scheduled task). The commit-and-push-per-phase discipline preserved every completed phase. After-incident recovery procedure (state verify → mirror parity → orphan container cleanup → resume from last commit) is documented in the incident file.

What Agents SHOULD Do

- Keep sessions alive: prefer short, bounded operations over indefinite waits
- For builds/tests longer than a few minutes, use background tasks where possible
- Monitor system load and avoid pushing the host to resource exhaustion
- If a container build or gradle build takes a long time, warn the user and use `--no-daemon` to prevent Gradle daemon from holding locks across suspends

Docker / Podman Specific Notes

- Container builds and long-running containers do NOT normally cause host suspend

- However, filling the disk with layer caches or consuming all CPU for extended periods can trigger thermal throttling or watchdog timeouts on some systems
- Always clean up old images/containers after builds to avoid disk pressure

§6.X — Container-Submodule Emulator Wiring Mandate (inherited 2026-05-13, per §6.F)

Inherited verbatim from parent Lava `/CLAUDE.md` §6.X (added 2026-05-13 in response to the operator's twenty-first §6.L invocation: "when we rely / depend on emulator(s) needed for the testing of the System, make sure we boot up Container running Android emulator in it using ours Containers Submodule. It is supported and it works, it just need proper connecting into the flows."). Every Android emulator instance MUST execute INSIDE a podman/docker container managed by `submodules/containers/`. Host-direct emulator launches are permitted for workstation iteration only; the constitutional gate run (release tagging, real-device verification) MUST go through the container-bound path. `pkg/runtime/` brings the container up; `pkg/emulator/` orchestrates the AVD lifecycle inside it. §6.X-debt tracks the wiring implementation owed to the Containers submodule. This submodule MAY add stricter rules but MUST NOT relax.

§6.AG — Containers-Submodule-Driven Emulators Mandate (added 2026-05-31, §6.L 68th invocation)

See parent Lava `/CLAUDE.md` §6.AG. Every test needing an Android device (Compose UI Challenge Tests, `connectedAndroidTest`, `-PdeviceTests=true`, the §6.AE matrix, any emulator-bearing gate) MUST obtain it from an emulator instance brought up + managed by the `vasic-digital/Containers` submodule (`submodules/containers/` `pkg/emulator/` + `cmd/emulator-matrix`), via `scripts/run-challenge-matrix.sh` / `scripts/run-emulator-tests.sh`. The device is a **cold-booted emulator AVD, NEVER a live/physical ADB device** (physical devices are presumed in use by other projects and MUST NOT be targeted). Per-OS implementation (the §6.X acceleration model): Linux x86_64 → emulator INSIDE a podman/docker container (`--device /dev/kvm`); macOS/arm64 → host-direct+HVF cold-boot (a Linux container cannot reach `/dev/kvm` — absent in the podman VM — nor HVF, a host-only API; this is the §6.X-resolved macOS gate path, still Containers-`emulator-matrix`-driven, still an emulator, NOT a live device); Windows → host-direct+WHPX. No live-device fallback — a test redirected to

a physical `adb` device because an emulator was slow/unavailable is a §6.AG violation; the test is OWED, not silently redirected (§6.Z/§6.J). Project-level clause per operator instruction + §11.4.35 (consumer-side, NOT the shared `constitution/` submodule). `Classification`: project-specific.

§6.AC — Comprehensive Non-Fatal Telemetry Mandate (added 2026-05-14)

Every catch / error / fallback / unexpected-state path on every distributable artifact MUST record a non-fatal telemetry event with sufficient context to triage the failure remotely. **Android client:** call `analytics.recordNonFatal(throwable, context)` (for throwables) or `analytics.recordWarning(message, context)` (for non-throwable warnings) at every meaningful error site. The `AnalyticsTracker` interface in `lava.common.analytics` is the canonical entry; under the hood it routes to Firebase Crashlytics's non-fatal channel. Cancellation throwables (CancellationException + wrappers) are filtered automatically as benign teardown noise. **lava-api-go:** equivalent `observability.RecordNonFatal(ctx, err, attrs)` helper emits structured WARNING/ERROR log + optional Crashlytics REST bridge when `LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true`. **Mandatory context attributes:** `feature` / `module`, `operation`, `error_class`, `error_message` (truncated 1024 chars, never include credentials per §6.H), and per-platform extras (`screen` for Android, `endpoint` + `request_id` for Go). **Forbidden:** silent fallbacks without telemetry (use `// no-telemetry: <reason>` to opt out explicitly), recording credentials/tokens/cookies/PII unredacted. §6.AC-debt is open: Detekt rule for Kotlin + Go-vet check for Go that flag `try/catch` blocks lacking the telemetry call; pre-push hook integration; documented but not yet mechanically enforced.

Forensic anchor: 2026-05-14 operator directive after 1.2.21-1041 stage-1 verification: "Add comprehensive Crashlytics non-fatals recording all over the apps and API so we can track in the background all warnings, issues and unexpected situations!" The §6.AB completeness checklist catches discrimination gaps in tests; §6.AC is the runtime equivalent — every error path that fires in production MUST surface to telemetry so the operator can triage real-user failures remotely.

Inheritance: applies recursively to every submodule's distributable artifacts and every new artifact added to the project.

§6.AB — Anti-Bluff Test-Suite Reinforcement (27th §6.L invocation, added 2026-05-14)

Every existing test + Challenge MUST be auditable for the anti-bluff property "would this test fail if the user-visible feature broke in the way a real user would notice?" Per-feature anti-bluff completeness checklist: rendering correctness (not just rendering presence — assert dominant color matches expected hue, not just RGB-variance > N), state-machine completeness (negative tests for forbidden transitions: back-from-Welcome MUST NOT mark onboarding complete; reaching home MUST require ≥1 successfully probed provider), gating logic (gate fires only on actual completion criterion, not on side effects of unrelated transitions). Bluff-hunt cadence escalation: every Crashlytics-or-operator-reported defect not caught by an existing test triggers a 5-file defect-driven bluff-hunt of adjacent tests, recorded under `.lava-ci-evidence/bluff-hunt/<date>-defect-driven-<slug>.json`. The §6.J-spirit discrimination test for every Challenge Test: before declared "covers the feature", the author MUST construct a deliberately-broken-but-non-crashing version of the production code and confirm the Challenge Test FAILS — because the 1.2.19 crash was the easy case (§6.Z catches it), the 1.2.20 white-icon + gate-bypass are the hard cases that this clause exists to prevent.

Forensic anchor: 2026-05-14 1.2.20-1040 debug installed on Galaxy S23 Ultra surfaced (a) Welcome brand mark renders as white placeholder (Compose `Icon` applies `LocalContentColor` tint, designed for monochrome glyphs) and (b) onboarding "complete" gate fires on Welcome back-press (`OnboardingViewModel.onBackStep()` Welcome branch posts `Finish`; `MainActivity` writes `setOnboardingComplete(true)` on `Finish`). Both passed all existing tests. Operator: "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

Inheritance: applies recursively to every submodule, every feature, every new artifact added to the project.

§6.AA — Two-Stage Distribute Mandate (Debug First, Release After Verification, added 2026-05-14)

When an artifact has both a debug and a release variant, distribute MUST happen in TWO STAGES with operator-confirmed verification between them. Stage 1: `firebase-distribute.sh --debug-only` distributes the debug APK to the `.dev`-suffixed app ID; operator (or designated tester) verifies the **Firebase-distributed**

debug APK on the failure-surface device class. Stage 2: ONLY AFTER written debug verification, `firebase-distribute.sh --release-only` distributes the release APK; §6.Z evidence file appended with release-stage section. No combined distribute permitted by default — that requires explicit per-cycle operator authorization recorded in the evidence file. The R8 / minification surprise class (§6.Z forensic anchor was a release-only `painterResource()` rejection) is the load-bearing reason: staging surfaces non-R8 bugs at debug's smaller blast radius AND isolates R8-specific failures to the release stage. §6.AA-debt is open: `scripts/firebase-distribute.sh` default flip to `--debug-only` + refusal of out-of-order `--release-only` + paired `last-version-{debug,release}` per-channel are documented but not yet mechanically enforced. Forensic anchor: 2026-05-14 operator: "for purposes like this one we shall distribute via Firebase DEV / DEBUG version only. Once we try it, you continue and once all verified you distribute RELEASE too!"

Inheritance: applies recursively to every submodule's distributable artifacts and every new artifact added to the project.

§6.Z — Anti-Bluff Distribute Guard (added 2026-05-14, TWENTY-SIXTH §6.L invocation)

No artifact may be distributed (Firebase App Distribution, Google Play Store release, container image push, lava-api-go binary release, any future distributable channel) UNLESS the corresponding Compose UI Challenge Tests (or per-artifact equivalent end-to-end tests) have been **EXECUTED — not source-compiled, EXECUTED** — against a real device or emulator running the EXACT artifact about to be distributed, AND have **passed**. Pre-distribute test-evidence file required at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>-test-evidence.{md,json}` with matching commit SHA, timestamp within 24h, and `BUILD SUCCESSFUL` verbatim in the captured gradle output for the per-channel mandatory test set (Android: at minimum C00 cold-start + C01 launch + every Cn whose target file appears in the cycle's diff). Cold-start verification (C00) is the load-bearing canary — skipping it for any reason is a §6.Z violation. There is no "this is a small change" exception (forensic anchor: 1.2.19-1039 was a one-line drawable swap that crashed every device on cold launch). §6.Z-debt is open: pre-distribute test-evidence gate (firebase-distribute.sh Phase 1 Gate 6) + pre-push hook check are documented but not yet mechanically enforced.

Forensic anchor: 2026-05-14 operator's 26th §6.L invocation, in response to a Crashlytics report on Lava-Android-1.2.19-1039: "Application crashes when we open it on Samsung Galaxy S23 Ultra with Android 16. Check Crashlytics, there should be entries.

Fix this and re-distribute! Another point, how come the build wasnt tested? Anti-bluff policy MUST BE ENFORCED ALWAYS!!!" The agent (this assistant) had distributed 1.2.19-1039 without executing C24/C25/C26 against an emulator — citing the darwin/arm64 §6.X-debt as the blocker, but §6.X-debt blocks LAN reachability of running APIs (mDNS broadcasts through the podman VM), NOT the running of Compose UI tests against a connected emulator. The operator's Pixel_7_Pro / Pixel_8 / Pixel_9_Pro AVDs WERE available. Distribute proceeded without test execution. Crashlytics issue `40a62f97a5c65abb56142b4ca2c37eeb` (FATAL, `painterResource()` rejecting a `<layer-list>` drawable in `LavaIcons.AppIcon`) hit every cold launch; C26 would have caught it on first emulator boot.

Inheritance: applies recursively to every submodule's distributable artifacts and every new artifact added to the project. Submodule constitutions MAY add stricter rules but MUST NOT relax this clause.

§6.Y — Post-Distribution Version Bump Mandate (added 2026-05-14, TWENTY-FIFTH §6.L invocation)

After every successful distribution of any artifact (Android APK via Firebase App Distribution, Google Play Store release, container image push, lava-api-go binary release, any future distributable artifact), the FIRST commit in the new development cycle that touches code MUST bump the artifact's `versionCode` integer (and the per-artifact equivalent for non-Android targets — see clause 2 of root CLAUDE.md §6.Y). The `versionName` semver MUST be bumped too when the changes warrant a user-visible version change (patch for bug fix, minor for feature, major for breaking change; held only for internal refactor / docs / test-only with a one-line rationale in the bump commit body). The bump-first commit MAY be version-bump-only OR contain the bump as the first hunk of a feature commit; bump-only is preferred for `git log` discoverability.

§6.Y-debt is open: pre-push hook + `scripts/check-constitution.sh` enforcement of the bump-first ordering is documented but not yet mechanically enforced. Until §6.Y-debt closes, reviewers MUST manually verify bump-first ordering on every commit that lands after a distribute. The forensic anchor: 2026-05-14 operator directive immediately after the 1.2.18-1038 Firebase distribute success: "after every application's distribution ... before any new changes have been applied to the product version code MUST BE increased to proper buildable target - API or Application! If it is required and it makes sense version MUST BE increased as well!" Inheritance: applies recursively to every submodule's

distributable artifacts and every new artifact added to the project. Submodule constitutions MAY add stricter rules but MUST NOT relax this clause.

Universal Action-Prefix Recognition (LAYER 1, §11.4.140)

When a user prompt's FIRST non-blank line starts with an uppercase action token followed by `::` (grammar `^([A-Z][A-Z0-9_]*)\s*::\s`, or the namespaced `PREFIX::ACTION ::` form): look the token up in `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); if registered, REPLACE the `ACTION_NAME ::` prefix with that action's `expansion` text, apply its `rules`, and execute the REMAINDER of the prompt under the expanded instruction (expand-then-rescan). Built-in: `BACKGROUND ::` → run the rest in the background, in parallel with all main work streams, subagent-driven, with rock-solid anti-bluff captured evidence. Unknown tokens are not actions — execute verbatim. The registry is the single source of truth (add actions as rows, not by editing this block). See root `CLAUDE.md` §6.AJ. Classification: universal.